

COMPUTACIÓ PARAL·LELA I MASSIVA - 1.2

Multiplicació de matrius disperses amb OpenMP

Nom: Jordà Coca Rodríguez

Curs: 2025 - 2026

Índex

- 1. Enunciat
- 2. Anàlisi de les dependències
- 3. Implementació paral·lela
- 4. Metodologia d'execució
- 5. Resultats
- 6. Conclusions

1. Enunciat

L'objectiu de la pràctica és paral·lelitzar amb OpenMP un algorisme de multiplicació de matrius disperses per $N = 8000$. El programa seqüencial original calcula tres variants: matriu A dispersa per matriu B densa, matriu A dispersa per matriu B dispersa i, finalment, una versió on A, B i C es guarden en format dispers.

Segons l'enunciat, els temps seqüencials de referència amb -O3 són aproximadament 16,2 s a Teen i 48,5 s a Orca. La mitjana harmònica mínima del speedup ha de ser 3,5 a Teen i 5 a Orca. A més, la versió paral·lela ha de donar el mateix resultat que la seqüencial.

2. Anàlisi de les dependències

En aquest apartat es descriuen les parts principals del codi i es justifica quines zones es poden paral·lelitzar. S'ha mantingut el criteri de no modificar la generació aleatòria inicial ni les ordenacions, ja que aquestes parts són auxiliars i no concentren el cost principal del programa.

2.1 Funcions auxiliars de comparació

Les funcions `cmp_fil` i `cmp_col` s'utilitzen com a comparadors del `qsort`. La primera ordena els elements dispersos per fila i la segona per columna. Aquest pas és necessari perquè les matrius disperses es guarden en vectors lineals de structs `tmd`, on cada element conté les coordenades i el valor no nul.

No s'ha paral·lelitzat aquesta part. Són funcions de suport cridades pel `qsort` i no representen el coll d'ampolla principal del programa.

2.2 Generació de matrius

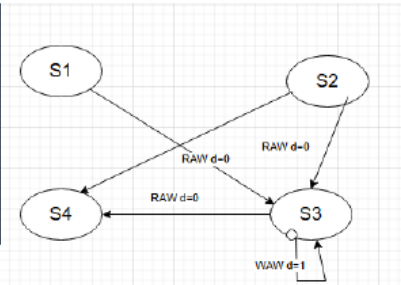
La generació de les matrius A i B es manté seqüencial. En aquesta fase es generen posicions aleatòries, s'eviten repeticions i es guarden els valors tant en format dens com en format dispers. Paral·lelitzar aquesta part podria alterar l'ordre de generació dels nombres aleatoris i, per tant, modificar el comportament respecte de la versió original.

2.3 Bucle 1: construcció de jBD

El primer bucle construeix l'array `jBD`, que fa d'índex d'accés ràpid a les columnes de la matriu dispersa B. Com que BD està ordenada per columnes, `jBD[j]` apunta a la primera posició de BD que correspon a la columna `j` o a una columna posterior.

Aquest bucle es manté seqüencial perquè la variable `k` té una dependència acumulativa entre iteracions. Cada iteració depèn del valor de `k` que deixa la iteració anterior, de manera que apareix una dependència `loop-carried`. Per això no és una zona adequada per a una paral·lelització directa.

```
k=0;           S1
for (j=0; j<N+1; j++) S2
{
    while (k < ND && j>BD[k].j) S3
        k++;      S4
    jBD[j] = k;    S5
}
```



2.4 Bucle 2: A dispersa per B densa

En aquest bucle es calcula C1 multiplicant la matriu A en format dispers pel format dens de B. Per a cada columna i de B es recorren els elements no nuls d'A i s'acumula el resultat en C1.

```
#pragma omp parallel for private(k)
for (i = 0; i < N; i++)
    for (k = 0; k < ND; k++)
        C1[AD[k].i][i] += AD[k].v * B[AD[k].j][i];
```

La paral·lelització es fa sobre el bucle exterior de i. Cada thread treballa sobre una columna diferent de la matriu resultat C1. D'aquesta manera, les escriptures es fan sobre posicions diferents de C1 i no hi ha condicions de carrera entre threads.

2.5 Bucles 3 i 4: A dispersa per B dispersa amb resultat dens

Aquesta part calcula C2 utilitzant tant A com B en format dispers. Per fer-ho eficientment, per a cada columna i de B s'expandeix temporalment aquesta columna en el vector auxiliar VBcol. Això evita haver de buscar repetidament dins de BD durant el càlcul.

El bucle d'inicialització de VBcol té un cost molt petit i es deixa seqüencial. En canvi, el bucle principal sobre i es paral·lelitzat. Com que VBcol és un buffer temporal, es declara com a threadprivate i s'utilitza copyin per donar a cada thread una còpia inicialitzada del vector.

```
#pragma omp threadprivate(VBcol)
for (j = 0; j < N; j++)
    VBcol[j] = 0;

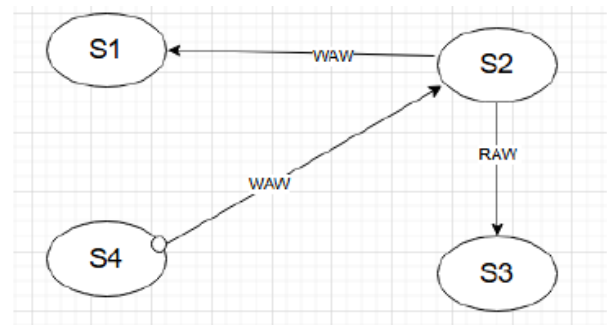
#pragma omp parallel for copyin(VBcol)
for (i = 0; i < N; i++) {
    for (k = jBD[i]; k < jBD[i + 1]; k++)
        VBcol[BD[k].i] = BD[k].v;

    for (k = 0; k < ND; k++)
        C2[AD[k].i][i] += AD[k].v * VBcol[AD[k].j];

    for (j = 0; j < N; j++)
        VBcol[j] = 0;
}
```

Aquesta paral·lelització és segura perquè cada iteració de i calcula una columna independent de C2. El buffer VBcol és privat per thread, per tant no hi ha escriptures simultànies sobre el mateix vector auxiliar.

```
for (j=0;j<N;j++)
    VBcol[j] = 0;                                S1
for(i=0;i<N;i++)
{
    for (k=jBD[i];k<jBD[i+1];k++)
        VBcol[BD[k].i] = BD[k].v;                S2
    for (k=0;k<ND;k++)
        C2[AD[k].i][i] += AD[k].v * VBcol[AD[k].j]; S3
    for (j=0;j<N;j++)
        VBcol[j] = 0;                            S4
}
```



2.6 Bucles 5 i 6: A i B disperses amb resultat C dispers

En aquesta tercera versió també es calcula el producte entre A i B disperses, però el resultat es comprimeix en el vector CD. Per cada columna es fa servir VBcol per expandir la columna de B i VCcol per acumular temporalment la columna de C abans de guardar només els valors no nuls.

```
neleC = 0;
#pragma omp threadprivate(VCcol, VBcol)
for (j = 0; j < N; j++)
    VBcol[j] = VCcol[j] = 0;

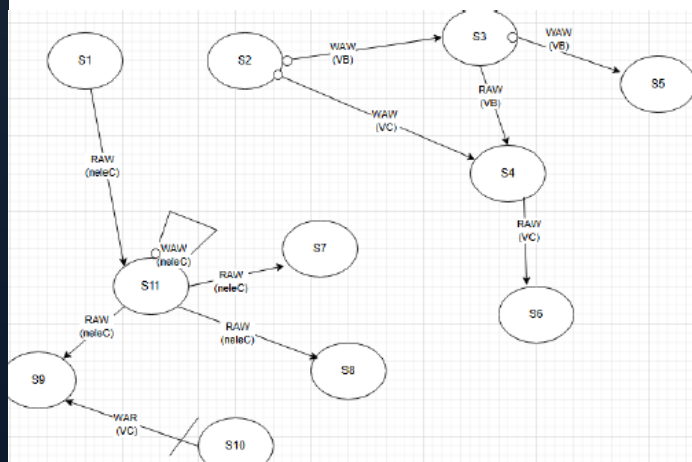
#pragma omp parallel for private(k, j) copyin(VCcol, VBcol)
for (i = 0; i < N; i++) {
    for (k = jBD[i]; k < jBD[i + 1]; k++)
        VBcol[BD[k].i] = BD[k].v;

    for (k = 0; k < ND; k++)
        VCcol[AD[k].i] += AD[k].v * VBcol[AD[k].j];

    #pragma omp critical
    for (j = 0; j < N; j++) {
        VBcol[j] = 0;
        if (VCcol[j]) {
            CD[neleC].i = j;
            CD[neleC].j = i;
            CD[neleC].v = VCcol[j];
            VCcol[j] = 0;
            neleC++;
        }
    }
}
```

En aquest cas cal protegir la fase de compressió amb una secció crítica, ja que tots els threads podrien escriure simultàniament sobre CD i actualitzar neleC. Sense aquesta sincronització apareixerien condicions de carrera i es podria alterar l'ordre o el contingut de la matriu dispersa resultant.

```
// ===== Inicialització =====
neleC=0;                                     S1
for (j=0;j<N;j++)                           S2
    VBcol[j] = VCcol[j] = 0;
// bucle per columnas
for(i=0;i<N;i++)
{
    // ===== Part 1 =====
    for (k=jBD[i];k<jBD[i+1];k++)
        VBcol[BD[k].i] = BD[k].v;           S3
    // ===== Part 2 =====
    for (k=0;k<ND;k++)
        VCcol[AD[k].i] += AD[k].v * VBcol[AD[k].j]; S4
    // ===== Part 3 =====
    for (j=0;j<N;j++)
    {
        // neteja vector de B[*][i]
        VBcol[j] = 0;                       S5
        // Compressió de C
        if (VCcol[j])                       S6
        {
            CD[neleC].i = j;                S7
            CD[neleC].j = i;                S8
            CD[neleC].v = VCcol[j];         S9
            VCcol[j] = 0;                   S10
            neleC++;                         S11
        }
    }
}
```



3. Implementació paral·lela

La implementació paral·lela manté la mateixa estructura general del programa seqüencial, però introdueix directives OpenMP en les parts amb més cost computacional. Les inicialitzacions petites, la generació aleatòria i la creació de jBD es mantenen seqüencials.

3.1 Pseudocodi general

```
Generar AD i A de forma seqüencial
Ordenar AD per files
Generar BD i B de forma seqüencial
Ordenar BD per columnes
Construir jBD de forma seqüencial

#pragma omp parallel for
Calcular C1 = AD * B

Inicialitzar VBcol del thread master
#pragma omp parallel for copyin(VBcol)
Calcular C2 = AD * BD amb resultat dens

Inicialitzar VBcol i VCcol del thread master
#pragma omp parallel for copyin(VBcol, VCcol)
Calcular CD = AD * BD amb resultat dispers
    #pragma omp critical
    Escriure en CD i actualitzar neleC

Comprovar C1 amb C2
Comprovar C1 amb CD
Calcular Suma
```

3.2 Ús de threadprivate i copyin

Els vectors VBcol i VCcol són buffers temporals utilitzats dins de cada iteració de columna. Si fossin compartits entre threads, diferents iteracions podrien modificar-los alhora i es produirien resultats incorrectes. Per això es fan threadprivate, de manera que cada thread disposa de la seva pròpia còpia.

La clàusula copyin s'utilitza perquè les còpies privades dels threads comencin amb el mateix estat inicial que la còpia del thread master. Això permet inicialitzar els buffers una vegada i evitar inicialitzacions innecessàries dins de totes les iteracions.

3.3 Secció crítica en la compressió de C

La part més delicada és la compressió de la matriu C dins de CD. Les variables CD i neleC són compartides. Per tant, cal una secció crítica quan s'afegeixen nous elements no nuls. Aquesta decisió redueix parcialment l'escalabilitat, però garanteix que la sortida sigui equivalent a la versió seqüencial.

4. Metodologia d'execució

Les execucions s'han fet forçant explícitament una única tasca Slurm i assignant t CPUs per tasca. Això és important perquè una execució OpenMP ha de correspondre a un sol procés amb t threads, i no a diverses tasques independents.

```
export OMP_NUM_THREADS=$t
export OMP_DYNAMIC=FALSE
export OMP_PLACES=cores
export OMP_PROC_BIND=spread
```

```
srun -p <particio> \
  --nodes=1 \
  --ntasks=1 \
  --cpus-per-task=$t \
  /usr/bin/time -p $BIN
```

Durant les proves inicials es va detectar que amb una assignació de CPUs lògiques es podia produir un comportament enganyós: per exemple, en el cas de 2 threads a Orca es van observar assignacions com 0 i 64, que corresponen al mateix core físic. Això feia que dos threads s'executessin sobre els dos fils lògics del mateix core mitjançant SMT/hyperthreading, i per tant el pas d'1 a 2 threads gairebé no millorava el rendiment.

Després d'analitzar Cpus_allowed_list i el mapatge de lscpu -e, es va ajustar l'execució perquè en els casos baixos s'utilitzessin cores físics diferents, per exemple CPU 0 i CPU 1 per a 2 threads. Aquesta correcció fa que les mesures siguin més representatives del paral·lelisme real del programa.

5. Resultats

Els temps s'han mesurat amb /usr/bin/time -p i s'ha utilitzat el camp real com a temps d'execució. El speedup s'ha calculat usant com a baseline el temps d'1 thread de la mateixa màquina: orca_1 per a Orca i teen_1 per a Teen.

Taula 1. Resultats a Orca

Threads	Temps real (s)	Speedup	Eficiència
1	48.63	1.00	100.0%
2	24.99	1.95	97.3%
4	13.24	3.67	91.8%
8	7.09	6.86	85.7%
16	4.05	12.01	75.0%
32	2.73	17.81	55.7%
64	2.20	22.10	34.5%
128	3.33	14.60	11.4%

La mitjana harmònica del speedup a Orca, considerant les execucions de 2 fins a 128 threads, és 5.91. Per tant, se supera el mínim demanat de 5.

Taula 2. Resultats a Teen

Threads	Temps real (s)	Speedup	Eficiència
1	16.27	1.00	100.0%
2	8.36	1.95	97.3%
4	4.52	3.60	90.0%
8	2.60	6.26	78.2%
16	2.48	6.56	41.0%
32	No executable	-	-

La mitjana harmònica del speedup a Teen, considerant les execucions vàlides de 2, 4, 8 i 16 threads, és 3.62. Per tant, se supera el mínim demanat de 3,5. L'execució amb 32 threads no s'ha pogut realitzar perquè el sistema retorna que el nombre de CPUs per node no es pot satisfer; això és coherent amb el fet que el node Teen utilitzat exposa 16 CPUs lògiques.

5.1 Gràfiques

A continuació es mostren les gràfiques de temps d'execució, speedup i eficiència.

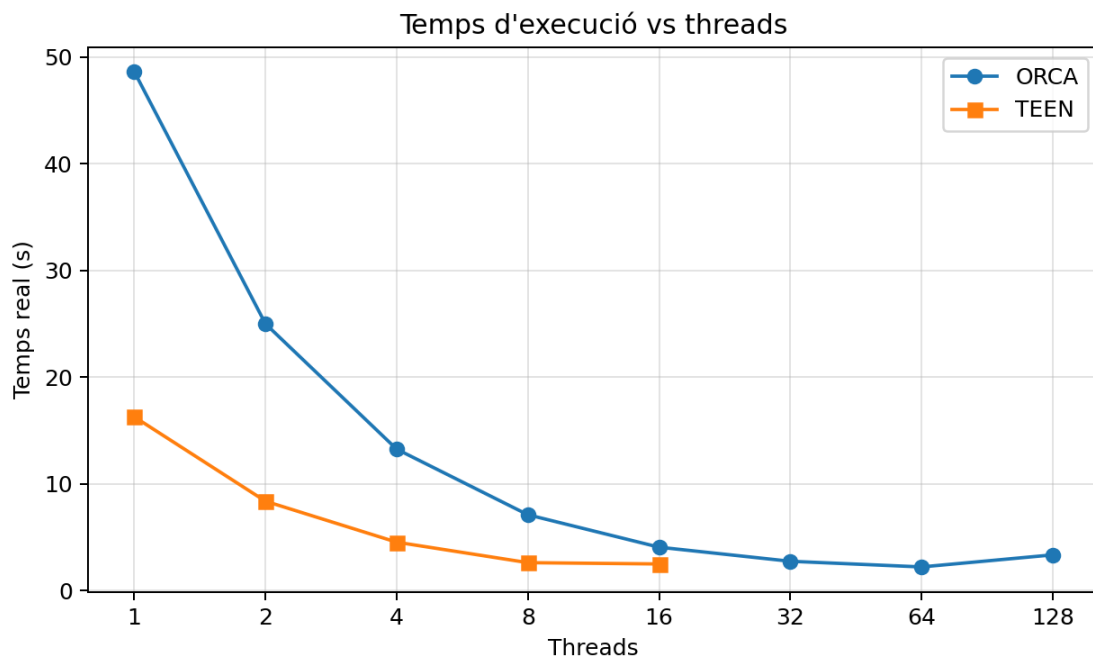


Figura 1. Temps d'execució vs threads.

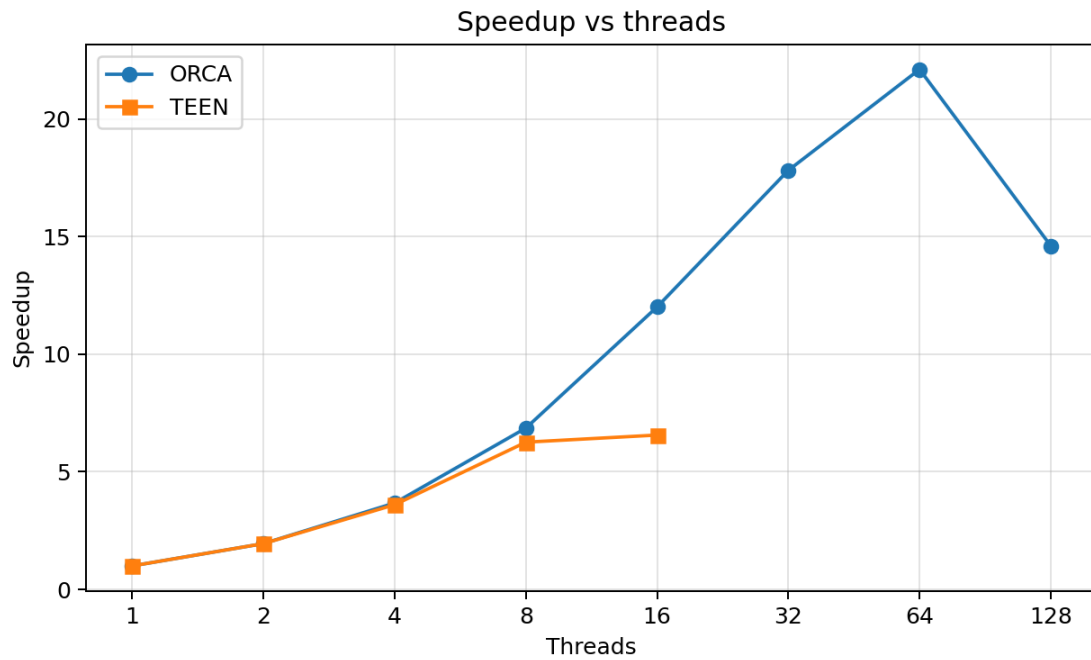


Figura 2. Speedup vs threads.

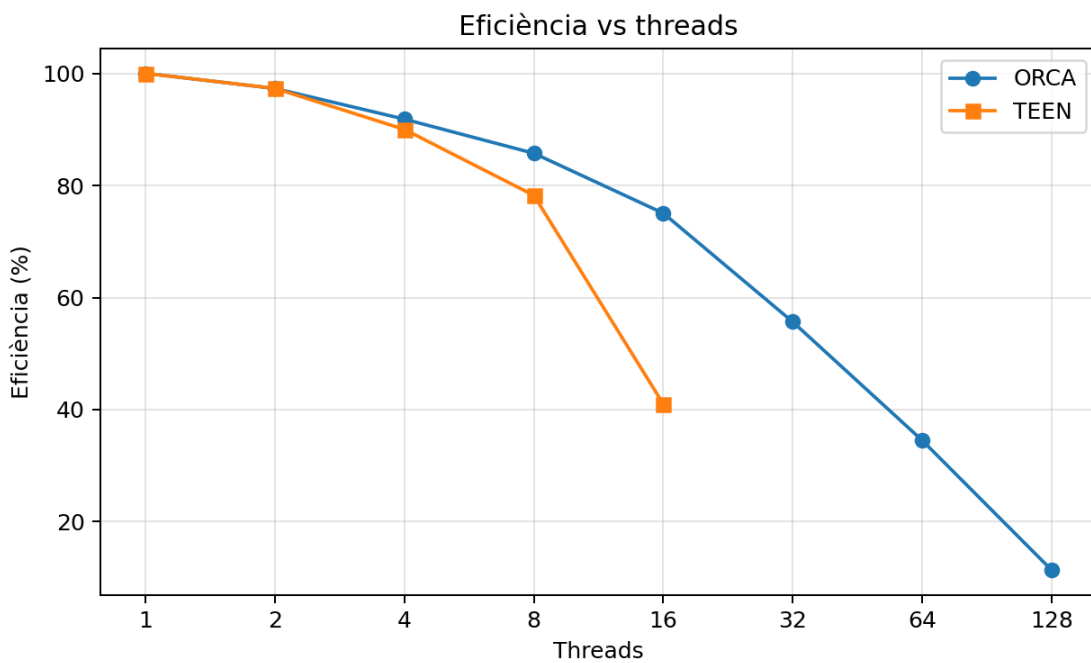


Figura 3. Eficiència vs threads.

5.2 Comprovació de la sortida

La versió paral·lela manté les comprovacions originals entre C1, C2 i CD. En les execucions finals no apareixen missatges de diferències, i els valors finals coincideixen amb els de la versió seqüencial.

Taula 3. Valors de sortida comprovats

Valor comprovat	Resultat
Elements no nuls de C	35248895
Suma dels elements de C	130625108751

6. Conclusions

La paral·lelització s'ha centrat en els bucles amb més cost computacional, especialment els que recorren totes les columnes i tots els elements no nuls de les matrius disperses. Les parts amb dependències acumulatives o amb cost molt petit s'han mantingut seqüencials.

A Orca s'obté una mitjana harmònica del speedup de 5,91, superior al mínim de 5. A Teen s'obté una mitjana harmònica de 3,62, superior al mínim de 3,5. Per tant, la versió paral·lela compleix els requisits de rendiment establerts a l'enunciat.

També s'ha verificat que la sortida és equivalent a la seqüencial. La principal dificultat experimental ha estat la correcta assignació de threads a cores físics, ja que una assignació inicial sobre fils lògics del mateix core feia que el pas d'1 a 2 threads gairebé no millorés el rendiment. Un cop identificat i corregit aquest punt, els resultats obtinguts són coherents amb l'arquitectura de les màquines utilitzades.